



Artwork: Susan Haejin Lee



How open source maintainers keep contributors—and

themselves—happy

IT'S NEVER TOO EARLY OR TOO LATE TO START THINKING ABOUT CONTRIBUTOR RELATIONS.

KLINT FINLEY // JULY 22, 2021

The ReadME Project amplifies the voices of the open source community: the maintainers, developers, and teams whose contributions move the world forward every day.

Contributor relations was a hot topic at the [Global Maintainer Summit](#) event hosted by GitHub on June 8-9 2021. The event organizers, GitHub developer advocate Kara Sowles and Brian Douglas, dedicated six sessions to the subject.



Maintainers do the heavy lifting in open source, spending countless hours fixing bugs, merging pull requests, triaging issues, and providing support. But community contributions are what really make open source special. From insightful suggestions to complex technical features, people from around the world contribute their skills and perspective to make open source software better. “All the work we do as maintainers is made possible with the help of our community of contributors,” [Tern](#) co-maintainer Rose Judge said during her talk at the Global Maintainer Summit. Indeed, this talent pool is the reason so many individuals and organizations open source software in the first place.

Maintainers struggling to [balance their open source work with the rest of their lives](#) can benefit greatly from the skills and perspectives of contributors. Contributors can help reduce maintainers' workloads by fixing bugs or implementing features, and it would be impossible for many projects to scale up to meet the needs of users without robust contributor communities.

But reviewing contributions, moderating discussions on issues and proposals, and managing communities creates new responsibilities for maintainers as contributor communities grow. Popular projects can attract so many prospective contributors that maintainers have to spend more time coordinating their activities than writing code. In her book on open source, *Working in Public*, researcher and author Nadia Eghbal points out that for many developers, contributing to a popular project is a mark of distinction. “I started to see the problem is not that there’s a dearth of people who want to contribute to an open source project, but rather that there are too many contributors—or they’re the wrong kind of contributors,” Eghbal wrote.

Finding ways to attract enough contributors to sustain projects while scaling-up contributor relations is one of the great challenges facing today’s maintainers.



Community building

Having too many contributors may sound like a welcome problem to new maintainers struggling to attract contributions.

During his keynote, [Kubernetes](#) co-founder Brendan Burns emphasized the need to make sure projects look as active as possible so that potential contributors don't assume they've been abandoned. That means pushing new code frequently.

But it's not enough to just *look* active. Burns also emphasized the need to respond quickly to questions and pull requests, a concern Judge highlighted as well. If you take too long, a potential user or contributor may move on. Organizations with sufficient resources can assign staff to respond to GitHub Issues, or even create an on-call schedule. Individual maintainers should just do their best. "This is not to say you shouldn't take a break," Judge said. "Just communicate to the community what they can expect while you're away."

Multiple speakers underscored the need to be welcoming. "People underestimate how important and how hard this is," Burns said. He points out that although you might be tired of answering the same questions over and over, new users and contributors won't know that. Snapping at someone for asking an innocent question will leave a bad impression of you and your project.

In her talk on contributor incentives, [FOSSASIA](#) founder Hong Phuc Dang pointed out that technology choices can play a big role in what projects people will want to contribute to. For example, she mentioned that using [React.js](#), instead of lesser-known user interface libraries, might help attract more talent since there is a huge base of developers who are already familiar with it and many more who want to learn it.

Cash payments are one obvious benefit that just about anyone will appreciate. But Dang emphasizes that recognition is also an important incentive for contributors. For example, FOSSASIA highlights contributions from individuals in social media posts, newsletters, and other communications so that everyone gets some recognition for the work they do. Giving contributors the opportunity to take on more responsibilities, whether that's merge access or having them help onboard new contributors, is another powerful motivator. "When people get more responsibilities they become more engaged and more excited about the project," Dang says.

Better contributions through automation

Triaging the work of contributors and maintaining a community is an enormous amount of work. Keeping open source projects, and their maintainers, healthy requires

a bigger investment in paying the people who do the work to keep the projects we all rely on running.

Rust Foundation Interim Executive Director Ashley Williams urged companies to hire more maintainers to work full-time on their projects. “There's no difference between investing in technology and investing in its people,” she said. The Rust Foundation is experimenting with a new spin on open source foundation sponsorship. Instead of donating money to the Rust Foundation, companies can become sponsors by hiring maintainers to work on Rust.

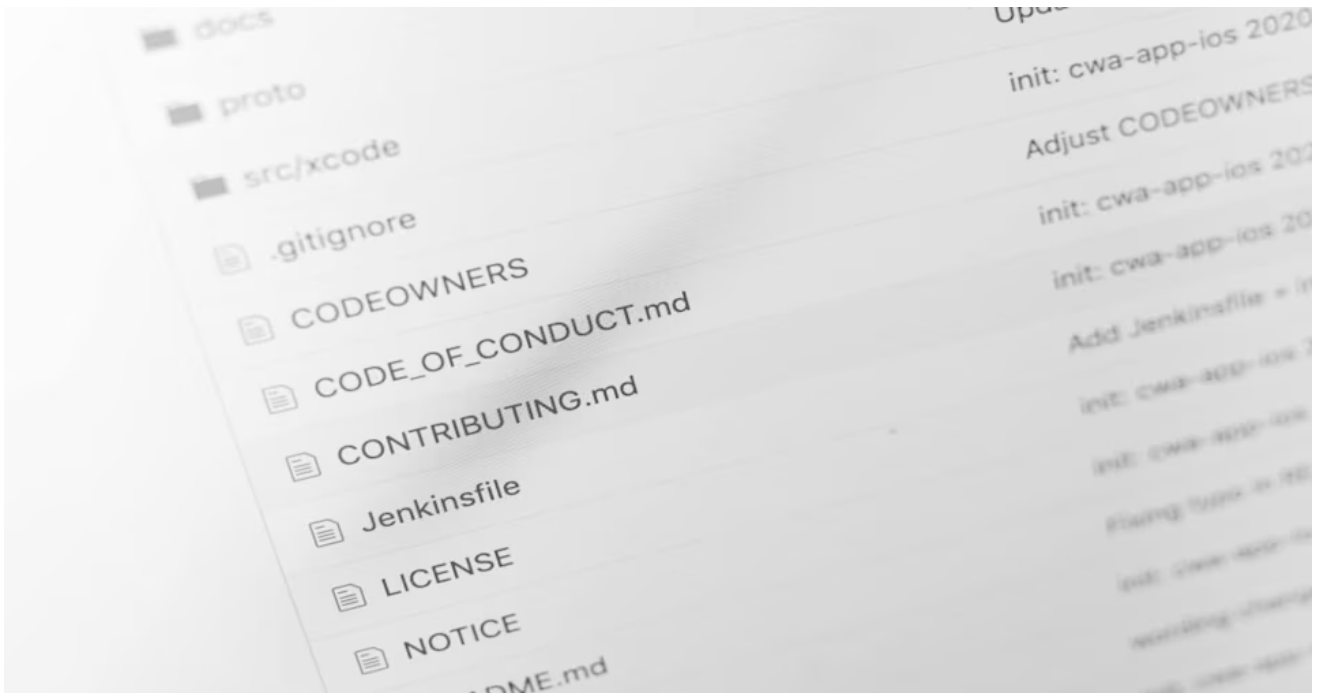
But even full-time maintainers can end up overwhelmed. “There's generally more work than anyone can actually do,” Paulus Schoutsen, maintainer of [Home Assistant](#), cautioned. Because there's a potentially infinite number of things maintainers can spend time on, the key is being strategic about what you do and, more importantly, *don't* do. Multiple speakers emphasized the importance of avoiding scope creep in projects. “Every extra line of code means more of a support burden,” said [Prometheus](#) maintainer Bartłomiej Płotka. New features need documentation and tests, and increase the number of questions users have.

Multiple speakers shared their approach to automation, a popular strategy to help maintainers manage contributions. Linters and code formatters are particular favorites. AutoFormatting also lowers barriers to entry since contributors are less likely to have code rejected over stylistic preferences. [Homebrew](#) maintainer Mike McQuaid noted that shifting to an automated code review process meant fewer arguments over minor details. “We weren't being accused, as maintainers, of being pedantic anymore,” he said.

Python core maintainer [Mariatta Wijaya](#) shared some of the custom bots her team uses to make reviewing contributions easier. For example, the “[Knights Who Say Ni](#)” verifies whether the contributor license agreement has been signed and, if not, blocks the submission until it has. Meanwhile, the “[Bedevere](#)” closes invalid pull requests and identifies problems such as missing issue numbers, while “[Miss-islington](#)” automatically backports approved pull requests.

Schoutsen recommended using [GitHub Actions](#) to automate tasks like locking old pull requests and automatically closing stale issues. Closing old pull requests and issues without resolving them might seem unfriendly, but Schoutsen points out that having lots of unresolved requests will make your project look like it isn't being maintained. And it could make it harder to respond to new requests in a timely fashion. “Keep your

plate clean,” he said. “If issues pile up, you're less likely to look at them. So just close the old ones. If it's important, it will come up again.”



Setting boundaries

As McQuaid notes, however, you can't automate everything. Some things, like offering praise and feedback, still need the human touch.

One of those things is saying “no” to contributions. Judge outlined several reasons you might have to reject a pull request, feature request, or other proposal. For example, a proposed change might introduce bugs, generate technical debt, or inflate the scope of your project. Sometimes you just won't have time to implement a feature request.

Saying no can be hard, [NgRX](#) maintainer Brandon Roberts noted in his talk, especially if you're suffering from imposter syndrome and worry that you might not make the right call. “Part of me didn't want to turn down contributions,” he said. “It's a challenge to put that work in.”

But, as Judge says: “You're the maintainer for a reason. Sometimes the answer is no and you shouldn't feel guilty for it.”

Roberts recommended defining a clear vision for your project to make it easier to make tough decisions—an idea echoed by several speakers. The NgRX team creates design documents to communicate the scope and vision of the project to users and

contributors. It's easier to say "no" when someone proposes a change that is outside the defined scope of a project.

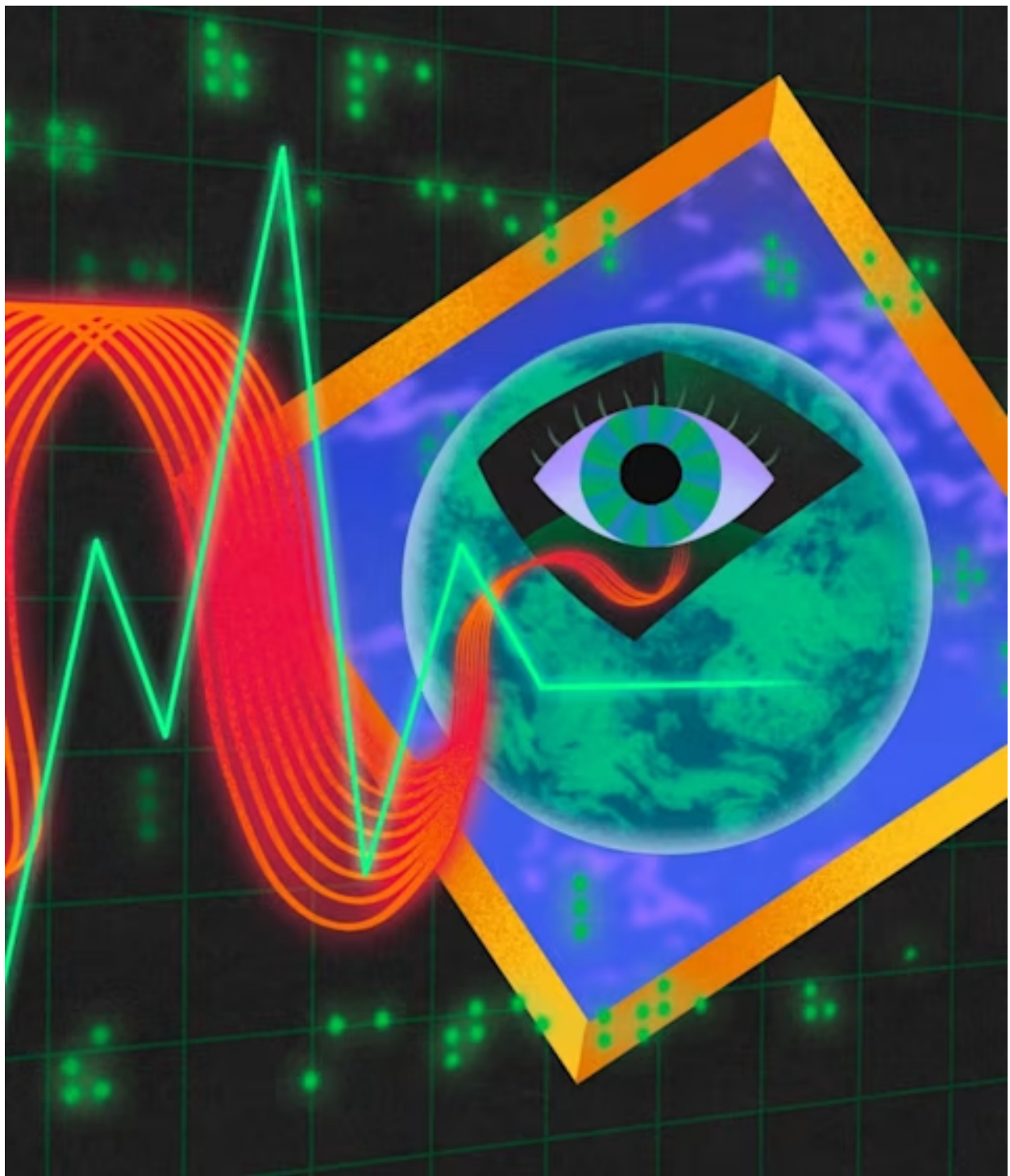
Judge emphasized that it's always important to thank people for their work, even if you're saying no to a contribution. It's important not to make rejections personal. "You are rejecting the proposal, not the person behind it," she said. When possible, try to phrase a rejection as an opportunity to say yes. For example, if you don't have time to implement a change, suggest they make the change themselves. "99% of the time they'll say no, but they might surprise you and it gives them a chance to become part of the solution," she says.

Once again, response time is key. The longer you take to respond, the more likely it is you'll discourage future participation. Just because someone is proposing something unworkable now doesn't mean they won't come back with a better idea in the future.

There are a few pieces of contributor relations that are applicable to all projects. "Be kind" and "take care of yourself and your people" are some of the most important and universal principles of open source. Beyond that, every maintainer will need to find the strategies that work best for their project.

You can [watch more talks from the summit](#) to learn more. By sharing their experience through events like these, maintainers are creating a body of knowledge to help each other build better contributor communities. It's yet another way the open source community is coming together to solve a common problem.

More stories



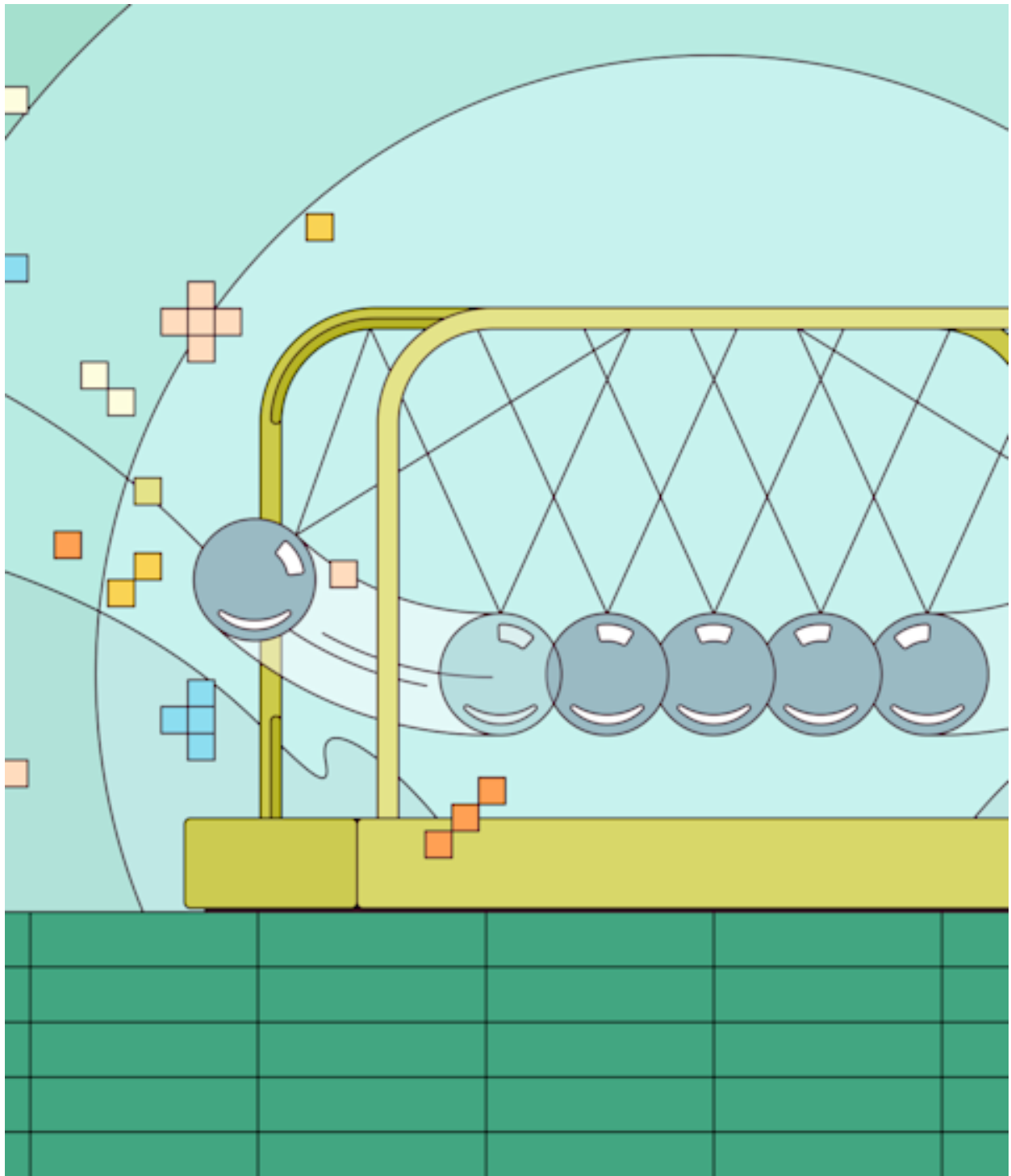
Coding accessibility: Building autonomy with AI

FEATURED ARTICLE



Is Laravel the happiest developer community on the planet?

FEATURED ARTICLE



TypeScript and the dawn of gradual types

FEATURED ARTICLE

About The ReadME Project

Coding is usually seen as a solitary activity, but it's actually the world's largest community effort led by open source maintainers, contributors, and teams. These unsung heroes put in long hours to build software, fix issues, field questions, and manage communities.

The ReadME Project is part of GitHub's ongoing effort to amplify the voices of the developer community. It's an evolving space to engage with the community and explore the stories, challenges, technology, and culture that surround the world of open source.

FOLLOW US:     

NOMINATE A DEVELOPER >

Nominate inspiring developers and projects you think we should feature in The ReadME Project.

SUPPORT THE COMMUNITY >

Recognize developers working behind the scenes and help open source projects get the resources they need.